Journal of Systems Architecture 154 (2024) 103220

Contents lists available at ScienceDirect

Journal of Systems Architecture

journal homepage: www.elsevier.com/locate/sysarc

A framework for static analysis and verification of low-level RTOS code

Vignesh Manjunath [a,b,*], Marcel Baunach [b]

[a] Pro2Future GmbH, Inffeldgasse 25F, Graz, 8010, Austria
[b] Graz University of Technology, Inffeldgasse 16, Graz, 8010, Austria

ARTICLE INFO

Dataset link: https://dx.doi.org/10.6084/m9.figshare.25657482

Keywords:
RTOS
WCET
Low-level code verification
Formal methods
Static analysis
Software reliability

ABSTRACT

Modern embedded software development uses model-based methods to support long-term maintenance, portability, and correctness. A growing trend is to use formal methods to create software models and verify their correctness against requirement specifications. However, modeling and verifying low-level Real-Time Operating Systems (RTOS) or Basic Software (BSW) code sequences remains a major challenge, as it requires correctness against the internal hardware behavior and timing. To ensure this correctness, we need formal models of the complex hardware architecture, and due to the increased model complexity, the verification can lead to a state space explosion.

In this paper, we mitigate these challenges by using an existing static Worst-Case Execution Time (WCET) analysis tool, OTAWA, for microarchitecture analysis. We use the intermediate results of the WCET analysis as input to our process, which verifies the correctness of the low-level implementations against the runtime effects of the hardware (e.g., synchronization dependencies, memory race conditions) and analyzes the timing and performance of the low-level code with respect to the data hazards in the pipeline. After successful verification, the results can be used in a formal method environment to model and verify the low-level code for correctness against the timing and requirement specifications. We demonstrate the proposed framework by analyzing and verifying the low-level context switch sequence of a classic AUTOSAR-based RTOS and the kernel startup sequence of FreeRTOS for correctness against hardware effects in the AURIX TriCore architecture. In addition, we show an empirical evaluation of our framework to examine the scalability, performance, and state space.

1. Introduction

An emerging practice in embedded software development is to use Model-Based Development (MBD) to model and verify software against functional and non-functional requirements. One such approach is to use formal methods to model and mathematically verify the correctness of the software. This reduces the need for rigorous testing strategies (e.g., black box testing) and promotes long-term maintenance, portability, and dependability.

In general, modeling and verification of high-level (hardware-independent) specifications of the application or Basic Software (BSW) is feasible, but this is not the case for their low-level or hardware-dependent counterparts, such as context switching or system initialization. These parts are implemented manually in assembly language and are subject to runtime effects (e.g., memory race conditions, synchronization dependencies) and performance bottlenecks (e.g., data hazards) caused by the hardware architecture. In addition, for high-level code, the compiler usually considers the internal hardware behavior when producing the assembly code, whereas this is not the case for manually implemented low-level code sequences. To successfully verify low-level or hardware-dependent code, we need to ensure its correctness against runtime effects in addition to the correctness against requirement specifications. Moreover, these low-level code sequences are time-critical and executed frequently. Therefore, we need to analyze the code for performance bottlenecks caused by hardware effects such as pipeline data hazards and the execution time of the low-level code.

The works [1–3] propose using formal methods to model and verify the low-level implementations of RTOS or BSW. However, they did not consider the impact of the hardware architecture on performance, timing, and the behavior of the low-level code during verification, as this requires formal models of the complex hardware architecture (e.g., Instruction Set Architecture (ISA), pipeline, memory). This leads to a significant increase in the model complexity. From a verification perspective, combining such a detailed hardware model with software models in the verification process can lead to a huge overall system model. Consequently, this can result in state space explosion [4], making verification infeasible.

This work addresses the outlined challenges through three key ideas: First, we mitigate the need for formal models of hardware by

* Corresponding author at: Pro2Future GmbH, Inffeldgasse 25F, Graz, 8010, Austria.
E-mail addresses: vignesh.manjunath@pro2future.at (V. Manjunath), baunach@tugraz.at (M. Baunach).

https://doi.org/10.1016/j.sysarc.2024.103220
Received 31 January 2024; Received in revised form 20 April 2024; Accepted 26 June 2024
Available online 2 July 2024
1383-7621/© 2024 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

using a static WCET analysis tool, OTAWA [5] as a microarchitecture analyzer. Second, we avoid the need to analyze hardware behavior in the formal method environment by performing verification using the intermediate results of the WCET analysis. The verification ensures that the behavior of low-level code is free from side effects caused by hardware effects such as structural hazards and synchronization dependencies. Third, we use the intermediate results of WCET analysis to analyze the execution times, and performance bottlenecks in the low-level code caused by the data hazards in the pipeline. Using intermediate results instead of the final results of the WCET analyzer suits our approach to consider all execution paths in the program and all possible hardware behaviors (e.g., cache hit/miss, pipeline behavior) for verification, performance, and timing analysis.

Once we successfully analyze and verify the low-level code behavior against the hardware architecture, we can use the results to construct formal models of the low-level code. Then, we can combine these models with the high-level (hardware-independent) models to verify correctness against the functional and non-functional software requirement specifications. The details on constructing the formal models of low-level code, the verification of software requirement specifications, and the optimization of low-level code by reducing the identified performance bottlenecks are considered future work and beyond the scope of this paper.

Contributions. We present and demonstrate a novel framework that enables performance analysis and verification of manually implemented low-level RTOS or BSW code sequences against internal hardware effects. We use OTAWA to analyze the compiled low-level code and extract intermediate results of the WCET analysis. Our main contributions in this paper are: (1) we define the verification specifications for synchronization dependencies and memory race conditions and formalize them as mathematical constraints; (2) we define the specifications to detect performance bottlenecks caused by data hazards in the pipeline and formalize them as constraints; (3) we present our principal component, the verification and performance analysis process that uses these constraints on intermediate results of the WCET analysis (in OTAWA) to identify performance bottlenecks and verify the side-effect-free behavior of the low-level code corresponding to the internal hardware effects; (4) using our process, we successfully analyzed and verified the manually implemented low-level context switch and kernel startup code sequences of two RTOSes.

We organize the remainder of the paper as follows: Section 2 gives a general background on the tools and methods used for the proposed framework. Section 3 describes the framework and the underlying methodology for performing verification and performance analysis. Then, in Section 4, we demonstrate our framework by analyzing and verifying the real-world context switch low-level code of a classic AUTomotive Open System ARchitecture (AUTOSAR) [6] based RTOS and the kernel startup code of FreeRTOS [7] for the AURIX TriCore TC275T architecture. Section 5 discusses the empirical evaluation of our verification and performance analysis. We comment on the reusability of the framework in Section 6. We discuss the related work in Section 7. Finally, in Section 8, we conclude the paper and give an outlook on future work.

2. Background

In this section, we briefly outline the tool OTAWA used in our method and provide insight into the AURIX TriCore architecture TC275T, the underlying hardware architecture for our framework demonstration.

2.1. The OTAWA framework

OTAWA [5] is a static WCET analysis framework based on abstract interpretation [8]. Fig. 1 provides a brief overview of the different analyses (denoted by No.) involved in the WCET analysis by OTAWA.

The framework parses and disassembles an input Executable and Linkable Format (ELF) file using the given hardware architecture specification (ISA, memory, pipeline). Then, ❶ the program analyses such as disassembly, program flow analysis, loop bound analysis, etc., are performed to graphically represent the program structure. The result is a set of Control Flow Graphs, one for each function or subroutine implementation. A Control Flow Graph (CFG) contains Basic Blocks as nodes connected by edges based on program flow. A BB is a sequence of instructions which always starts with the first instruction (e.g., a jump target) and always ends with the last instruction of the type *Control* such as `jmp`, `call`, `ret` (i.e., a BB cannot be entered/exited at any intermediate instruction). Also, the CFG of a function caller is connected to the CFG of the called function via the BB that invokes the call and the BB on return (depending on the program flow).

During program analyses, the target BB might be unidentifiable from a respective BB (e.g., due to an indirect function call). Hence, the program flow becomes indefinable, and in the corresponding CFG, the target BB is marked as `Unknown BB`. To resolve this, the target BB address can be provided to OTAWA as a program invariant using *flow fact information* input.

After the program analyses, ❷ a sequence of static analyses involving caches, memory access, and branch prediction, as defined in the WCET analysis script, is performed for all BBs (in all CFGs) of the program. These analyses use abstract interpretation [8] and perform data flow analysis for all execution paths in the program. As a result, for each BB in the program, they derive an abstract CPU state (referred to as STATE_OUT) in which registers and memory may contain constants or ranges of values based on Circular Linear Progression (CLP) [9] rather than a concrete CPU state in which registers and memory contain only constant values. The STATE_OUT of a BB serves as the STATE_IN for the next BB in the program. To facilitate the analysis, we can optionally specify the input abstract state of registers at desired program points as CLP expressions using *flow fact information*.

❸ The timing analysis uses the abstract states to compute the timing and order of instruction execution taking, the predecessor BBs (one per execution path), instruction dependencies, pipeline behavior, and memory access events (e.g., cache hit/miss) into account. On completion, a set of eXecution Graphs [10] is generated for each BB and its immediate predecessor BB (according to the program flow). The XGraph describes the possible states of instruction execution in the Central Processing Unit (CPU) pipeline and the transitions between them while considering the hardware behavior. The number of XGraphs per BB depends on the number of predecessor BBs (one for every execution path to the current BB) and the number of dynamic memory access events (e.g., cache hit/miss). This way, the analysis covers all execution paths and all possible hardware behaviors (e.g., pipeline and memory access events). The number of XGraphs ($N_{XGraphs}$) per BB is given by

$$N_{XGraphs} = \sum_{i=1}^{B} \left(\sum_{j=0}^{P_i} 2^{N_{ij}} \right) \tag{1}$$

where: B is the number of BBs in the program, P_i is the number of immediate predecessor(s) of BB_i, and $2^{N_{ij}}$ is the total number of combinations of dynamic memory access events per BB and its predecessor, i.e., if there are three dynamic memory access events ($N_{ij} = 3$) for a BB and its predecessor, then eight XGraphs are generated for that BB.

Finally, ❹ OTAWA computes the end WCET timing by formulating the WCET computation as an optimization problem and solving it using Integer Linear Programming (ILP) [11]. The OTAWA framework calculates all timing information in CPU cycles.

2.2. The AURIX TriCore architecture: TC275T

AURIX TriCore is a common architecture in embedded automotive systems that fulfills safety-critical requirements and achieves a high level of dependability of automotive software. Furthermore, OTAWA

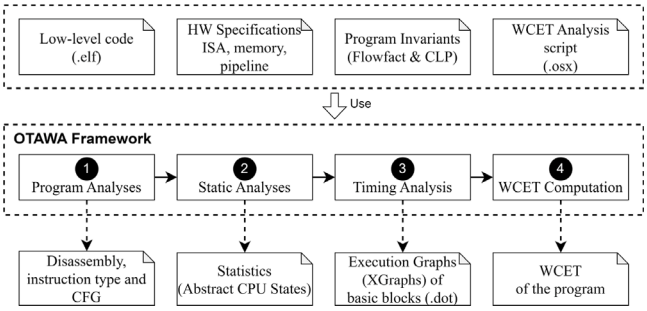


Fig. 1. Overview of the WCET analysis framework OTAWA.

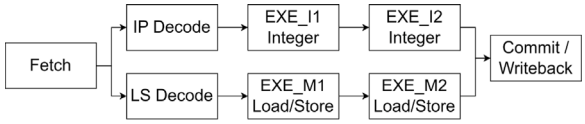


Fig. 2. Abstract model of the AURIX TriCore TC1.6E pipeline [15].

implements a hardware model of the AURIX TriCore TC275T architecture and verifies it in the work [12]. Hence, we assume that the hardware model is correct and choose TC275T as the underlying hardware architecture for our framework demonstration.

The TC275T is a 32-bit super-scalar processor with one E-core (TC1.6E) and two P-cores (TC1.6P) implementing the TC1.6 architecture [13]. The TC1.6E is a scalar processor and issues one instruction per clock cycle. The TC1.6E pipeline architecture has: a shared fetch unit, an execution unit containing an integer pipeline, and a load/store pipeline, each with three stages (one decode stage and two execute stages). The TC1.6P is a super-scalar processor and has: a fetch unit with issue FIFO (First In, First Out) buffering up to six instructions in the program order, an execution unit containing an integer pipeline, a load/store pipeline, and a loop pipeline each with three stages (one decode stage and two execute stages). All three pipelines in TC1.6P operate in parallel, permitting up to three instructions to be executed in one clock cycle.

The instructions of type *int, multiply and accumulate, etc.,* are executed in the integer pipeline, while the instructions of type *control, load, store, address arithmetic, context-related, sync, etc.,* are executed in the load/store pipeline. In TC1.6E, the loop-related instructions (loop, jned, jnei, etc.) are executed in the load/store pipeline, whereas in TC1.6P, these instructions (loop-related) are executed in the loop pipeline. To demonstrate our framework, we verify the low-level context switch and kernel startup code sequences in TC1.6E.

The General Purpose Register (GPR) file in TC275T contains sixteen 32-bit data registers (`D0-D15`) and sixteen 32-bit address registers (`A0-A15`). The CPU context comprises the GPR, Program Counter (PC), Program Status Word (PSW), and Previous Context Information (PCXI) registers. The context is subdivided into upper context comprising the registers `A10-A15` and `D8-D15`, `PSW`, and `PCXI`, and lower context comprising the registers `A2-A7`, `D0-D7`, `A11` (return address), and `PCXI`.

Figs. 2 and 3 shows the abstract pipeline model of the AURIX TriCore TC1.6E and TC1.6P respectively. The commit stage is used as a reference in OTAWA to mark the completion of instruction execution (it does not exist in the hardware architecture). Further details regarding instruction timings, pipeline, and CPU Subsystem can be found in the TC275T user manual [14].

3. Methodology

Fig. 4 illustrates the proposed methodology to analyze timing and performance, and verify the correctness of low-level code against internal hardware effects. We explain our methodology in two parts:

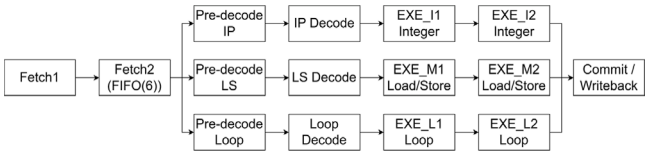


Fig. 3. Abstract model of the AURIX TriCore TC1.6P pipeline [15].

Section 3.1 gives an insight into the intermediate results of the WCET analysis and the information we extract. Section 3.2 describes how we use the information from the intermediate results in our principal component, the verification and performance analysis process, and the results obtained at the end of the process.

3.1. Analysis of low-level code

First, we compile the low-level assembly code into an ELF executable using the target-specific compiler. Next, we use OTAWA to analyze the ELF using the hardware specifications, flow fact information, and the WCET analysis script. We then extract the following intermediate results generated during the different phases of the WCET analysis:

(1) The program information, including the CFGs, the disassembly, and the instruction type (*control, load, store*), are generated during the program analyses phase. We use the disassembly and the instruction type information to analyze performance bottlenecks in the low-level code.

(2) The statistics information logged during the static analyses phase contains the abstract CPU states of the BBs (STATE_IN and STATE_OUT). In our approach, we extract the STATE_IN of all the BBs and use it in memory race condition check.

(3) The XGraph contains the timing information, order of instruction execution, pipeline behavior, and inter-dependencies between the instructions of a given BB, the predecessor BB, and the pipeline stages. Fig. 5 shows the XGraph of a BB generated by OTAWA during the WCET analysis of a sample code sequence for the TC275T architecture.

The instructions `I0` and `I1` (black nodes) in the XGraph (refer to Fig. 5) belong to the predecessor BB, and the instructions `I2` and `I3` (green nodes) belong to the current BB for which the XGraph is generated. The instructions in the two BBs are numbered according to their order of execution and correspond to the *program order* (sequence of instructions in the order specified by the program) in the case of the TC275T architecture.

The XGraph shows five pipeline stages: Fetch (`FE`), Decode (`DE`), and two execution stages, namely `EXE_I1`, `EXE_I2` for the integer pipeline and `EXE_M1`, `EXE_M2` for the load/store pipeline. The last stage, Commit (`CM`), is a reference used by OTAWA to mark the completion of instruction execution. The `DE(I3)` in XGraph represents the node where the instruction `I3` is in the `Decode` stage of the pipeline. The edges between two nodes imply a dependency between the respective instructions. The edges are of two types: The dashed edges (- →) indicate that the instructions are located in the same cache line. The solid edges (→) indicate data or address register dependencies, *program order*, and *pipeline order* (order of instruction execution in the pipeline).

The execution time of every node is indicated between <> for the respective node, i.e., `EXE_I1(I2)<1>` denotes that `EXE_I1` stage of the instruction `I2` takes one cycle to execute. The arrival time of every node in an XGraph is based on the *maximum(arrival time + execution time)* of incoming edges. For example, there are three incoming edges for the node `EXE_I1(I2)`, and the arrival time is 19, which corresponds to the *pipeline order* edge from the node `DE(I2)`. The completion time of any node is the sum of its arrival and execution time, i.e., the completion time for `EXE_I1(I2)=19+1=20`.

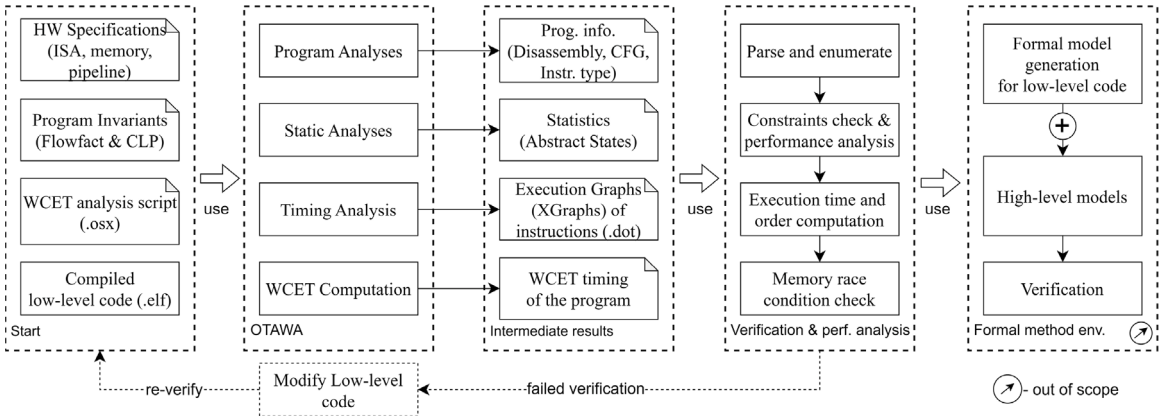


Fig. 4. Methodology for the verification of low-level code.

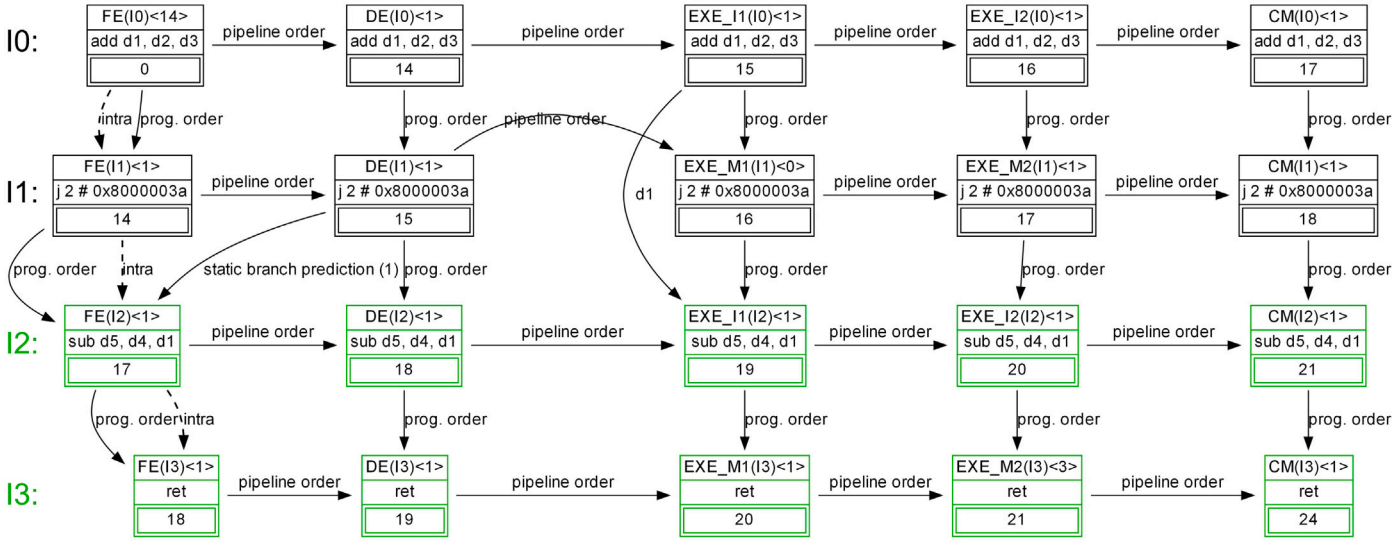


Fig. 5. XGraph of a BB generated by OTAWA for a sample code sequence in the TC275T architecture. □ - instructions in predecessor BB, □ - instructions in current BB. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

For mathematical comprehensiveness, we use the notation for execution time as the member `T` of the node (i.e. `EXE_I1(I2).T=1`), arrival time as the member `A` of the node (i.e. `EXE_I1(I2).A=19`) and completion time as the member `C` of the node (i.e. `EXE_I1(I2).C=20`).

The execution time of the instruction I_n is given by

$$I_n.E = CM(I_n).C - CM(I_{n-1}).C \tag{2}$$

where: I_{n-1} is the predecessor of instruction I_n in the *program order*. For example, the execution time of the instruction `I2` is `22-19=3`.

The execution time of the basic block BB_n is given by

$$BB_n.E = CM(I_{BB_n}).C - CM(I_{BB_{n-1}}).C \tag{3}$$

where: I_{BB_n} is the last instruction of BB_n and $I_{BB_{n-1}}$ is the last instruction of BB_{n-1} (predecessor of BB_n) in the *program order*. For example, the execution time of the BB (in green) in the XGraph is `(24+1)-(18+1)=6`.

We can determine the order of instruction execution based on the completion time of the `CM` stage. The XGraph provides instruction-level timing and dependencies information, which are important in our approach. Furthermore, the XGraphs of a BB include the predecessor BBs and the possible dynamic memory access events (e.g., cache hit/miss). Since we use XGraph information in our approach, we implicitly cover all execution paths and all possible dynamic hardware behaviors (e.g., pipeline and memory access events).

(4) WCET of the program is obtained from the analysis log. The formal method environment can use the end WCET result and the timing information from XGraph to verify timing properties.

Our approach uses the intermediate results listed above to analyze performance bottlenecks and verify the low-level code for correctness against hardware effects.

3.2. Verification and performance analysis process

Many hardware effects can cause undesirable low-level code behavior or performance bottleneck. We focus on two main goals: First, to ensure that the low-level code does not exhibit undesirable behavior due to internal hardware effects. Second, to analyze performance bottlenecks in the low-level code. Accordingly, we define the side-effect-free behavior of the low-level code corresponding to hardware effects such as synchronization dependencies and memory race conditions in the TriCore architecture as verification specifications (Sn) below:

(S1) **Data synchronization dependency:** The execution of instructions related to context handling or cache maintenance should be completed before a data/memory access is initiated by the next instruction(s) in the *program order*. Otherwise, impending data/memory access must be synchronized with the completion of context handling or cache maintenance.

Table 1
Constraints for verifying the low-level code.

Spec. Sn	Case	Program order		Spec. Hn	Constraint
		I_n	I_{n+1}		
(S1)	1	DSYNC	MTCR	(H1)	$(I_n = MTCR) \wedge (I_{n-1} = DSYNC)$
	2	co./ca.[a]	DSYNC	(H1)	$EXE_M2(I_n).C \leq EXE_M1(I_{n+2}).A$
	3	co./ca.[a]	instr.[b]	(H1)	$EXE_M2(I_n).C \leq EXE_M1(I_{n+1}).A$
(S2)	1	MTCR	ISYNC	(H2)	$(I_n = MTCR) \wedge (I_{n+1} = ISYNC)$
	2	MTCR	instr.[c]	(H2)	$EXE_M2(I_n).C \leq FE(I_{n+1}).A$
(S3)	1	Load	Store	(H3)	$EXE_M2(I_n).C \leq EXE_M1(I_{n+1}).A$
	2	Store	Load		
	3	Store	Store		

[a] Context or cache related instruction.
[b] Any instruction except DSYNC.
[c] Any instruction except ISYNC.

(S2) **Instruction synchronization dependency:** The execution of instructions that cause a change in hardware architectural state (e.g., modifications of special function registers) should be completed before fetching the next instruction(s) in the *program order*. Otherwise, the fetch of impending instruction(s) must be synchronized with the completion of previous instructions.

(S3) **Memory race condition:** If two instructions access the same memory location through different registers, and at least one of the instructions modifies the data, the first access must be completed when the second access starts. Otherwise, a memory race condition may occur between the two instructions.

With reference to the TC275T user manual [14] and Infineon low-level drivers [16], and Section 2.2, we determine the internal hardware behavior relevant to the specifications (S1)-(S3) and list them as hardware specifications (Hn) below:

(H1) `DSYNC` in TriCore waits for previous data accesses to complete before allowing any further data/memory access instructions to execute. The `DSYNC` instruction should be executed: (1) before the `MTCR` instruction (modifies core special function registers like PCXI), (2) after context handling instructions (e.g., `SVLCX`, `RSLCX`), and (3) cache maintenance type instructions (e.g., `CACHEA.I`).

(H2) `ISYNC` instruction flushes the CPU pipeline once the previous instructions have completed their execution. The instructions that follow `ISYNC` are fetched again. `ISYNC` should be executed after `MTCR` instruction (only when accessing specific `CSFR` registers as listed in [14]) so that any change made by the `MTCR` instruction will be seen by the next instruction(s) in the *program order*.

(H3) For instructions executed in the load/store pipeline (e.g., *control, load, store*), the address resolution is performed in *execute stage 1* (EXE_M1), and memory access is done in *execute stage 2* (EXE_M2) of the pipeline.

Based on conditional reasoning, the internal hardware behavior (H1)-(H3), and the description of TriCore architecture, we formalize the verification specifications (S1)-(S3) into the set of mathematical constraints listed in Table 1. The constraints follow the notations of completion time `C` and arrival time `A` (refer to Section 3.1) and vary according to the instructions and their *program order* (I_n, I_{n+1}). For example, case 1 of (S1) means that an instruction I_n is DSYNC, and the next instruction in the *program order* I_{n+1} is MTCR. If the low-level code satisfies the constraints in Table 1, then the verification is successful. Otherwise, we have to analyze the code and might need to modify and verify it again. Note: A constraint check FAIL for a (overly) pessimistic constraint might indicate a potentially undesirable behavior of the code. In such cases, further analysis is required to determine whether the code really exhibits an undesirable behavior and modify it accordingly.

To address our second goal, we define below the specifications (Pn) to analyze the performance bottlenecks caused by data hazards in the pipeline:

Table 2
Constraints for analyzing performance bottlenecks in low-level code.

Spec. Pn	Hazard	Constraint
(P1)	$RAW_{I,J}$	$(W_I \cap R_J) \neq \emptyset$
(P2)	$WAR_{I,J}$	$(R_I \cap W_J) \neq \emptyset$
(P3)	$WAW_{I,J}$	$(W_I \cap W_J) \neq \emptyset$

Note: `I` and `J` are instructions where `I` precedes `J`, `W` is the set of registers written by the respective instruction, and `R` is the set of registers read by the respective instruction.

(P1) **Read After Write (RAW) hazard:** The instruction that reads a register, follows the instruction that writes to the same register.

(P2) **Write After Read (WAR) hazard:** The instruction that writes to a register, follows the instruction that reads the same register.

(P3) **Write After Write (WAW) hazard:** The instruction that writes to a data register, follows the instruction that writes to the same register.

We use conditional reasoning and set theory to formalize the specifications (P1)-(P3) into the set of mathematical constraints listed in Table 2. The constraints differ for each data hazard (RAW, WAR, WAW), and two instructions constitute a performance bottleneck if they satisfy any of these constraints.

Next, we explain our principal component, the verification and performance analysis process. The intermediate results of WCET analysis (refer to Section 3.1) serve as the input to our process, and we illustrate the sequence of steps (denoted by (No.)) in Fig. 6.

In (1), we parse the CFG files, disassembly, and statistics information and extract the program flow, instructions, BBs, CFGs, and instruction type information. We then enumerate the program information (CFG, BB, instructions) as indexed data structures in (2). The enumeration allows us to index CFGs, BBs, and instructions based on the CFG ID, the address of the first instruction in the BB, and the address of the instruction, respectively. Next, in (3), we parse the XGraph files of every BB, and for each XGraph, we perform the constraint check, performance analysis, and memory race condition check.

In (4), to verify (S1) and (S2), we first search the instruction pairs (refer to Table 1: "program order" column) in the XGraph. For each identified instruction pair, we check the corresponding constraint in Table 1. Then, we log the results: the instruction pairs and its case, the timing information from XGraph (if applicable), and the results of the constraint check (PASS or FAIL).

In (5), to analyze (P1)-(P3), we first extract the set of registers read and written by the instructions in the XGraph. For each instruction pair in the XGraph, where one precedes the other, we check the constraints in Table 2. When an instruction pair satisfies a constraint, we log the type of data hazard and the instruction pair.

In real hardware, the data hazard does not happen between two dependent instructions when the first instruction completes its execution before the second one could start. So, in our analysis, for each identified

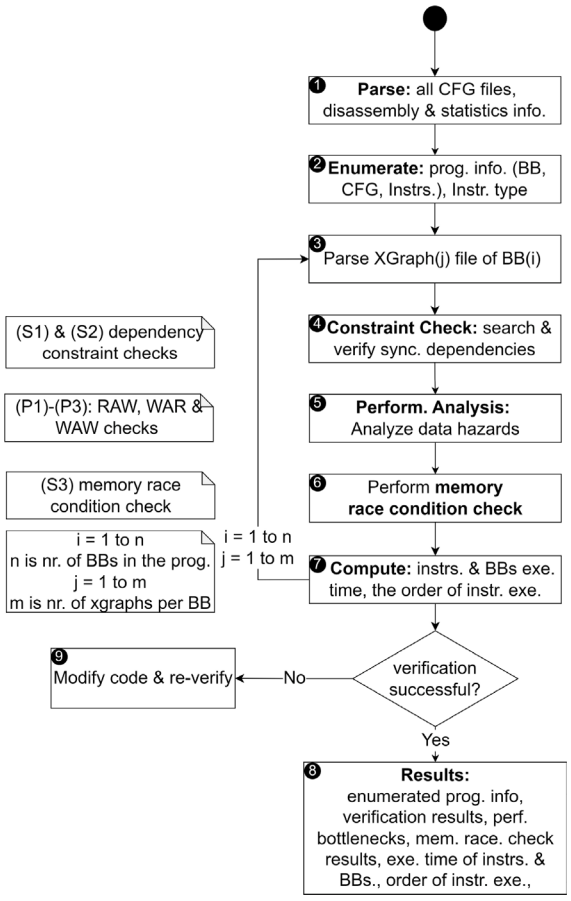


Fig. 6. Verification and performance analysis process flow.

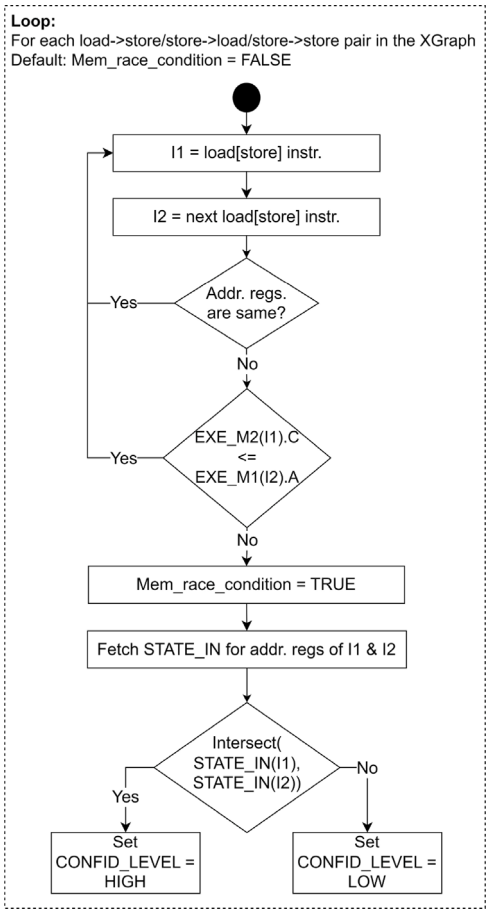


Fig. 7. Flow of the memory race condition check.

performance bottleneck, we analyze the likeliness of the data hazard based on the check: $\texttt{EXE_2(I}_m\texttt{).C} \leq \texttt{EXE_1(I}_n\texttt{).A}$ where: I_m and I_n are instructions, I_m precedes I_n, EXE_1 and EXE_2 corresponds to *execute stage* 1 and 2 of the respective pipeline (load/store or integer), and it follows the notations described in Section 3.1. If the above check passes, then, the first instruction had completed its execution, and the likeliness of the data hazard is LOW, otherwise, it is HIGH.

Next, in ❻, we check for memory race conditions using the statistics information and XGraphs, as shown in Fig. 7. For each *load* → *store* or *store* → *load* or *store* → *store* pair in the XGraph: we skip the check if the address registers are the same (since it is a performance bottleneck analyzed in ❺). If the address registers are different and they (the instructions) do not satisfy the constraint with respect to (S3) in Table 1, then the instructions may encounter a memory race and we set the memory race condition to TRUE (default value is FALSE). We then compare the abstract states (STATE_IN) of the respective address registers of both instructions. If the range of values in the abstract states overlap, then the instructions are highly likely to encounter memory race condition and we set the confidence level (CONFID_LEVEL) to HIGH. Otherwise, we set the CONFID_LEVEL to LOW.

The results of the memory race condition check indicate a possibility of memory race between two instructions. However, we need to know the functional logic or implementation details of the low-level code to determine if it really leads to a memory race condition. Finally, in ❼, we compute the execution time of the instructions and the BBs using Eqs. (2) and (3). The order of instruction execution is the same as the *program order* in the case of TC275T architecture. If the verification in ❹ is successful, then in ❽ we publish the results: (1) the program information (CFGs, BBs, instructions, program flow info.), (2) the verification results, (3) the identified performance bottlenecks, (4) memory race condition check results, (5) execution time of instructions and BBs, and (6) the order of instruction execution. Otherwise, ❾ we have to analyze the code and might need to modify it to fix the failed constraint check and verify it again. We perform the steps ❸–❼ for every XGraph and store the results. Therefore, the time taken by our process is proportional to the number of XGraphs given by Eq. (1).

We have implemented the verification and performance analysis process logic in Python 3.1. The XGraphs and the CFGs are represented in DOT language [17]. In DOT language, they are lexically specified as 'digraph'. The timing information in XGraph is formatted using HTML [18]. We parse them using Pydot Graphviz library [19] and use regular expressions to extract the information. We handle the data structures in our process using the Pandas Python data analysis library [20]. The artifacts provide further details on how to use our Python script, the script requirements, input files, and command line arguments.

We can use the results of our process (from ❽) to construct formal models of low-level code. We can then combine these low-level models with high-level models in the formal method environment to verify the requirement specifications. The construction of formal models, integration with high-level models, the verification of software requirement specifications, and the optimization of low-level code based on the performance bottlenecks are beyond the scope of this work. In the next section, we use our framework to analyze and verify a real-time low-level code for the TC275T architecture.

4. Demonstration

We experimentally evaluated our framework by analyzing and verifying the low-level code modules of a classic AUTOSAR-based RTOS

and FreeRTOS for the TC275T architecture (e.g., context switch, kernel startup, interrupt handlers). Here, we demonstrate our framework by analyzing and verifying two low-level code modules: (1) context switch and (2) kernel startup. The evaluation results of our framework are available in the artifacts, and we have included the information regarding artifacts at the end of the paper.

4.1. Context switch code of a classic AUTOSAR-based RTOS

We choose context switch code to demonstrate our framework since it is a safety-critical and real-world use case that must be manually implemented in low-level code. Moreover, context switch code is error-prone when porting RTOS to different hardware. Therefore, context switch is a more important and complex use case to analyze and verify for correctness against hardware effects.

```
1  j        OS_SystemCall    ;BB1
2
3  OS_SystemCall:    ;BB2
4  ld.bu    d14,[a8]36
5  mov      d0,#1
6  st.b     [a8]36, d0
7  call     OS_KernDummySyscall ;BB10
8
9  ld.a     a14,[a8]8    ;BB3
10 ld.a     a13,[a8]16
11 ld.a     a12,[a13]16
12 jne.a    a13, a14, _SaveAndSwitch
13
14 _WhichSwitch:     ;BB6
15 ld.bu    d15,[a12]8
16 jeq      d15,#2,_FreeAndSwitch
17
18 st.b     [a8]36,d14   ;BB9
19 rfe
20
21 _FreeAndSwitch:  ;BB7
22 rslcx
23 dsync
24 j        _Dispatcher
25
26 _SaveAndSwitch:  ;BB4
27 svlcx
28 dsync
29 mfcr    d15,#0xfe00
30 st.w    [a12]32,d15
31
32 _Dispatcher:     ;BB8
33 mov     d4,#0
34 dsync
35 mtcr    #0xfe00,d4
36 isync
37 ld.w    d4, [a14]0
38 st.w    [a13]0, d4
39 mov     d0,#0
40 st.b    [a8]36, d0
41 j       OS_Dispatch
42
43 OS_Dispatch:     ;BB5
44 mov     d15,1651
45 addih   d15,d15,199
46 dsync
47 mtcr    #0xfe00,d15
48 rslcx
49 dsync
50 rfe
```

Listing 1: Context switch code of a classic AUTOSAR-based RTOS. Note: The BB numbering corresponds to Fig. 8.

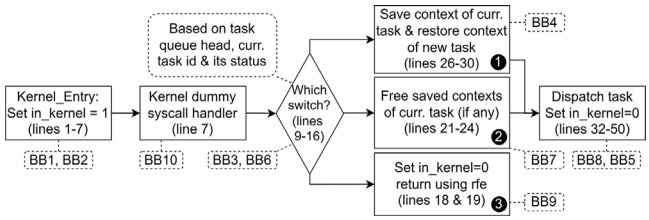


Fig. 8. Flow of the context switch code. Note: The line references and BB numbering corresponds to Listing 1.

Table 3
Results of ISYNC constraint check (S2) for context switch code.

Case	I1	I2	STG(I1).C	STG(I2).A	Result
1	(35)mtcr 0xfe00, d4	(36)isync	Not applic.	Not applic.	PASS
2	(47)mtcr 0xfe00, d15	(48)rslcx	23.0	19.0	**FAIL**

Note: The column *case* refers to the cases in Table 1. The columns `STG(I1).C` and `STG(I2).A` follow notations defined in Section 3.1 and `STG` refers to the pipeline stage with respect to the constraint in Table 1. We removed the instruction address and indicated the corresponding line of code (in Listing 1) in the columns `I1` and `I2`.

The context switch code is triggered on a syscall invocation (e.g., yield, sleep, waitForEvent). Since the functional logic of syscalls is usually a high-level implementation, we consider a dummy syscall for this demonstration. Listing 1 shows the context switch code, and Fig. 8 briefly visualizes the code execution flow, the corresponding lines of code in Listing 1, and the respective BBs.

We consider the following 3 cases in the context switch code: ❶ If the current task (the task that invoked the syscall) is not the same as the task that is scheduled to run next, then save the context of the current task, restore the context of the next task, and dispatch the next task. ❷ If the current task execution is complete and there is a new activation of the same task that is scheduled next, then free the contexts that were previously saved and dispatch the task. ❸ If the current task execution is not complete and the same task is scheduled to run, then just return using `rfe` (return from exception).

First, we compile the context switch code and analyze it using OTAWA. As a result, OTAWA generates two CFGs (one for the context switch code and one for the syscall), statistics information, and 65 XGraphs. The context switch CFG has 10 BBs, and we map them relative to the functional logic blocks in Fig. 8 and Listing 1. The syscall CFG has one BB, and it is implicitly referred to by BB10 in the main CFG. Now, we perform verification and performance analysis as described in Fig. 6. We parse the intermediate results of the WCET analysis and perform constraints check, performance analysis, memory race conditions check, and finally, compute the execution time of instructions and BBs.

The results indicate one constraint failure with respect to case 2 of (S2), 10 performance bottlenecks with likeliness of the data hazard to be HIGH, and two possible memory race conditions. The results imply that the verification is not successful. Here, we present only the results of the constraint check (S2) in Table 3 and the results of the memory race condition check in Table 4.

The constraint corresponding to the case 2 of instruction synchronization dependency (S2) failed since `EXE_M2(I1).C=23` is greater than `FE(I2).A=19`. The instruction `I2 RSLCX` attempts to restore the lower context before the instruction `I1 MTCR` updates the core special function register (`PCXI`) that points to the saved lower context. According to the specification (S2), in this case, we need to synchronize the fetch of `I2` with the completion of `I1`. Therefore, we add the `ISYNC` instruction after the `MTCR` instruction.

Based on the results in Table 4, we identify two possible memory race conditions with a `LOW` confidence level. According to the functional logic of the context switch code, the respective address registers in both cases point to different data structures. In the first result, the address register `A14` points to the task ready queue, and `A13` points

Table 4
Memory race condition check results of context switch code.

I1	I2	Mem. race condition	Confid. level
(37)ld.w d4, [a14]0	(38)st.w [a13]0, d4	TRUE	LOW
(37)ld.w d4, [a14]0	(40)st.b [a8]36, d0	TRUE	LOW

Note: We removed the instruction address and indicated the corresponding line of code in the columns I1 and I2.

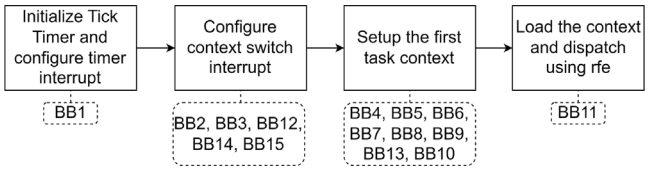


Fig. 9. Flow of the kernel startup code of FreeRTOS. Note: Every functional logic block is relatively mapped to the respective BBs in the program.

to the Task Control Block (TCB) of the current task. In the second result, A14 points to the task ready queue, and A8 points to the kernel data structure (in_kernel variable). Therefore, neither of the results causes memory race conditions, and no fixes are necessary.

Since we modified the code (by adding the ISYNC instruction), we re-verified the low-level code. The new constraint check results indicate that the verification is successful and the context switch code is side-effect-free with respect to the synchronization dependencies and memory race conditions. Note: The results of the memory race condition check still remain the same. However, as per the functional logic, we already know that they do not occur. Hence, we ignore the memory race condition check results.

We provide the program flow information and the execution time of BBs in Table 5. The execution time of BBs consists of a set of unordered discrete values computed during the analysis of the respective XGraphs. These values depend on the different execution paths (i.e., predecessor BB) to the BB and the number of dynamic memory access events. We can determine the program flow based on the BB ID and Successor BB ID columns. Since TC275T has no out-of-order execution, the order of instruction execution corresponds to the *program order* of instructions within the BB. As per the WCET analysis of the context switch code, the WCET is 282 CPU cycles.

4.2. Kernel startup code of FreeRTOS

To show that our approach is not limited to the low-level code sequences of an AUTOSAR-based RTOS, we demonstrate our framework with the kernel startup code of FreeRTOS. The kernel startup code is a system initialization routine that (1) initializes the tick timer and configures the timer interrupt, (2) configures the context switch interrupt, (3) sets up the first task's context, and finally, (4) loads the context and dispatches the task using rfe.

The source code is available in the artifacts, and Fig. 9 briefly visualizes the code execution flow. Similar to the analysis and verification of context switch code in Section 4.1, we first compile the kernel startup code and analyze it using OTAWA. As a result, OTAWA generates one CFG with 15 BBs, 374 XGraphs, and statistics information. We map the BBs relative to the functional logic block in Fig. 9. Next, we parse the intermediate results of the WCET analysis and perform constraints check, performance analysis, and memory race conditions check, and finally compute the execution time of instruction and BBs (refer to Fig. 6).

The results indicate three constraint check failures, 149 performance bottlenecks with the likeliness of the data hazard to be HIGH, and two possible memory race conditions. The verification is not successful, and the constraint check failures correspond to cases 1 and 3 of (S1). Table 6 presents only the results of the constraint check (S1).

The constraint check corresponding to case 1 of data synchronization dependency (S1) failed since the instruction I2 is MTCR and the instruction I1 is not DSYNC. According to (H1), the DSYNC instruction should be executed before the MTCR instruction. The constraint check corresponding to case 3 of (S1) failed since EXE_M2(I1).C=18 is greater than EXE_M1(I2).A=17. According to (S1), the execution of a context handling instruction I1 RSLCX (restores lower context) should be completed before the start of execution of instruction I2 RFE (restores upper context and returns from exception). Therefore, we need to add a DSYNC instruction before the MTCR instruction for case 1 constraint check failures and after the RSLCX instruction for the case 3 constraint check failure.

In Table 7, we present the results of the memory race condition check. We identify two possible memory race conditions. The instructions' address range corresponds to BB13, and its(BB13) functional logic is: get the lower context address from the task stack, increment the task stack pointer, get the critical nesting counter value of the task from the task stack, and increment the task stack pointer. According to the functional logic and code analysis, the respective address registers point to different memory locations. In the first result, A4 points to the critical nesting counter value of the task, and A2 points to the OS critical nesting counter variable. In the second result, A2 in the instruction I1 points to the OS critical nesting counter variable, and A15 in the instruction I2 points to the top of the task stack. Therefore, neither of the results causes memory race conditions, and no fixes are necessary.

To fix the (S1) constraint check failures, we add the DSYNC instructions accordingly. Then, we re-verify the code, and the new results indicate that the verification is successful and the kernel startup code is side-effect-free with respect to the synchronization dependencies. The memory race condition check results remain the same, and we ignore them since we already know that they do not occur as per the functional logic.

Table 8 provides the program flow information and the execution times of BBs in the program and follows the same format as Table 5. The WCET of the kernel startup code based on the WCET analysis is 538 CPU cycles.

The time to analyze and verify the context switch code was about two minutes, and the kernel startup code was about 30 min on a machine running Windows 10, 4-core Intel(R) Core(TM) i7 CPU @ 1.80 GHz, 16 GB DDR4 memory. For reference and reproducibility, we provided the following artifacts: intermediate results of WCET analysis, the final results (described in Section 3.2) of the initial and updated context switch and kernel startup code, and the source code of our process at https://doi.org/10.6084/m9.figshare.25657482.

5. Empirical evaluation

The intermediate results of the WCET analysis, i.e., program information, statistics, and XGraphs form the input state space examined by our verification and performance analysis process. The number of BBs and their immediate predecessor(s) are finite when the length of the program is finite. Only the number of XGraphs is dynamic due to the number of dynamic memory access events. OTAWA always limits the maximum number of dynamic memory access events for a given BB and its immediate predecessor (N_{ij}) to 16, resulting in a maximum of 65535 XGraphs (refer to Eq. (1)) for this BB and its immediate predecessor. However, this high number is unusual for a real-world, manually implemented low-level code. Additionally, we organize the intermediate results as indexed data structures, which supports us in examining the state space regardless of its size. This way, unlike conventional software verification approaches, the state space in our approach is bounded, always finite, and feasible to explore.

To assess the performance and scalability of our process, we performed an empirical evaluation using programs from the embench benchmark suite [21]. We performed the evaluation on a 4-core Intel®

Table 5
Program flow information & BB execution time of context switch code.

CFG	CFG ID	BB ID	BB start address	Successor BB ID	Execution time of BB (in CPU cycles)
core0_main	1	1	800000bc	2	(4, 17)
core0_main	1	2	80000034	10	(8, 21, 34, 10, 23, 36)
core0_main	1	3	80000042	6, 4	(7, 60, 11, 64)
core0_main	1	4	80000066	8	(8, 21, 9, 22)
core0_main	1	5	8000009a	Exit	(15, 13)
core0_main	1	6	80000052	9, 7	(2, 6)
core0_main	1	7	8000005c	8	(6, 19, 7, 20)
core0_main	1	8	80000074	5	(17, 29, 70, 82, 123, 135, 13, 77, 119,130, 24, 66, 14, 79, 120, 132, 26, 67)
core0_main	1	9	80000056	Exit	(5)
core0_main	1	10	OS_KernDummySyscall	3	Not applic.
OS_KernDummySyscall	2	1	800000ba	3 - CFG 1	(7, 3)

Table 6
Results of DSYNC constraint check (S1) for kernel startup code.

Case	I1	I2	STG(I1).C	STG(I2).A	Result
1	jnz d15, #0x800029da	mtcr 0xfe00, d0	Not applic.	Not applic.	**FAIL**
1	insert d15, d15, 0, 0, 7	mtcr 0xfe04, d15	Not applic.	Not applic.	**FAIL**
3	rslcx	rfe	18	17	**FAIL**

Note: The column *case* refers to the cases in Table 1. The columns `STG(I1).C` and `STG(I2).A` follow notations defined in Section 3.1 and `STG` refers to the pipeline stage with respect to the constraint in Table 1. We removed the instruction addresses to improve readability.

Table 7
Memory race condition check results of kernel startup code.

I1	I2	Mem. race condition	Confid. level
800029c4 ld.w d1, [a4]	800029c6 st.w [a2], d1	TRUE	LOW
800029c6 st.w [a2], d1	800029c8 ld.a a2, [a15]	TRUE	LOW

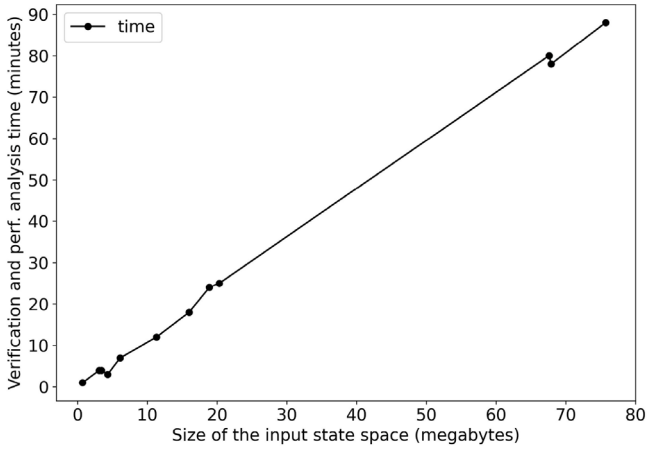


Fig. 10. Performance of the verification and performance analysis process.

Core™ i7 CPU @ 1.80 GHz with 16 GB DDR4 memory, running Windows 10.

We chose the size of input state space as a useful metric to compare the verification and performance analysis time of our process. This is because it (the size of input state space) takes into account: the complexity of the program (i.e. the number of CFGs, BBs, instructions and their dependencies), the size of abstract states (i.e. the size of statistics information), and all possible execution paths and dynamic memory access events (i.e. the number of XGraphs). Fig. 10 shows the performance of our process for different sizes of the input state space. Here, we can see that the verification and analysis time has a moderate linear relationship with the size of the input state space. This supports the usability claim of our approach in real projects.

6. Reusability

Our framework currently supports the analysis and verification of low-level code in the TC1.6E and TC1.6P cores of the AURIX TriCore architecture. For analysis and verification in TC1.6P, we modified the number of pipeline stages in the DOT files parsing module of the XGraphs. The constraint for case 2 of (S2) is modified as $EXE_M2(I_n).C \leq FE1(I_{n+1}).A$. Apart from these changes, the rest of our framework remains the same. We analyzed and verified the context switch code (see Section 4.1) for TC1.6P and provided the results in the artifacts.

We can extend our framework to other major embedded system architectures such as ARM and RISC-V since OTAWA already implements them. To extend our framework to a new hardware architecture, we expect the following steps: (1) determine the new hardware specifications (Hn) (refer to the corresponding user manuals, datasheets, and other architecture documents), (2) modify the constraints in Table 1 with respect to the pipeline stages, the order of execution (in-order or out-of-order), and the hardware behavior (derive new constraint cases if applicable), and (3) adapt the DOT files parsing module (i.e., modify regular expressions) in our process according to the ISA. We can reuse the rest of our framework across different architectures without any changes.

Completeness of the specifications: The specifications (S1)-(S3) and (P1)-(P3) form the basic set of properties that are critical for the low-level RTOS or BSW code during runtime. The specifications (S1)-(S3) and (P1)-(P3) are complete and sufficient to analyze and verify the low-level code for the hardware effects that we considered. The hardware specifications (H1)-(H3) with respect to the specifications (S1)-(S3) are complete according to the information in the user manual [14]. In general, the completeness of the specifications depends on the program being analyzed, the hardware effects considered, and the availability of hardware information (in datasheets and user manuals).

Table 8
Program flow information & BB execution time of kernel startup code.

CFG	CFG ID	BB ID	BB start address	Successor BB ID	Execution time of BB (in CPU cycles)
xPortStartScheduler	1	1	80002892	14, 2	(1, 22, 35, 87, 100, 88, 101, 34, 47, 48, 75)
xPortStartScheduler	1	2	800028c4	15	(5, 18, 4, 17)
xPortStartScheduler	1	3	8000296c	13	(5, 4)
xPortStartScheduler	1	4	800029da	5	(9, 22, 10, 23)
xPortStartScheduler	1	5	800029ee	6	(4, 6)
xPortStartScheduler	1	6	80002a06	9, 7	(3, 16, 2, 15, 6, 19)
xPortStartScheduler	1	7	800029f2	8	(10, 63, 11, 64)
xPortStartScheduler	1	8	80002a04	6	(5, 2, 1)
xPortStartScheduler	1	9	80002a0c	11	(5, 4)
xPortStartScheduler	1	10	800029d0	11	(8, 9)
xPortStartScheduler	1	11	80002a0e	Exit	(11, 24, 37, 9, 22, 35)
xPortStartScheduler	1	12	80002968	13	(5, 6)
xPortStartScheduler	1	13	8000296e	10, 4	(52, 64, 69, 82, 65, 63, 76, 56, 49, 61, 66, 79, 62, 60, 73, 53, 48, 78, 59, 72)
xPortStartScheduler	1	14	800028c0	15	(5, 18, 6, 19)
xPortStartScheduler	1	15	800028c6	12, 3	(67, 80, 85, 98, 111, 71, 84, 97, 78, 110, 91, 123, 75, 88, 101, 72, 64, 69, 82, 77, 95, 108, 68, 81, 94, 107, 120, 63, 76, 93, 74, 106, 87, 119)

7. Related work

There are a variety of approaches to analyze and verify low-level code targeting different objectives. The paper [22] proposes a sophisticated algorithm for analyzing data dependencies and memory race conditions in low-level code. However, unlike us, the method is limited to SPARC architectures and does not consider other hardware effects.

Similar to our framework, the work [23] uses abstract interpretation to perform static analysis of assembly code for DSP processors to detect runtime errors and analyze the accuracy of numerical computations. The works [24,25] analyze the data flow in x86 executables and derive an abstract state for registers and memory. In [26], a rule-based technique is used to verify the CFG of the low-level code for side effects due to discrepancies in opcodes or operands. However, these works analyze and verify assembly code at the semantic level and do not consider internal hardware behavior in their analyses.

The works [27,28] convert the low-level code into an Intermediate Representation (IR) and then perform a flow-sensitive analysis such as Value Set Analysis (VSA) to determine the state of registers and memories. The latter work [28] integrates with METAMOC [29] (a WCET tool with formal hardware models) to include hardware architecture in their analysis. Unlike our framework, these methods check the low-level code for program-level defects (e.g., buffer overflow, memory leaks) and do not analyze the runtime hardware effects.

The formal verification frameworks proposed in [2,3,30] focus on the correctness of interrupt safety properties in low-level RTOS code. Here, the authors convert the code to IR and avert the need for hardware models. In contrast to us, they perform a machine-specific semantic analysis and do not consider the internal behavior of the hardware. An Event-B [31] based approach for formal modeling and verification of low-level RTOS code was proposed in [1]. However, the authors did not consider the correctness of the low-level code against the internal effects of hardware.

The work [32] implements a hardware module to check for memory errors and invalid control flows in binary code. The hardware module verifies the program at the start and prevents it from loading if there are any issues. However, the approach does not verify whether the program is side-effect-free with respect to the internal hardware behavior (e.g., pipeline and cache) as we do.

The works Bedrock [33,34] and CertiKOS [35] propose verified low-level programming frameworks based on the Coq Proof Assistant [36]. Bedrock specifies low-level code in its intermediate language (Bedrock IL) and verifies memory safety and functional correctness. CertiKOS specifies kernel components using a layered methodology and verifies functional and contextual correctness. Unlike our framework, neither Bedrock nor CertiKOS considers internal hardware behavior and does not verify low-level code for correctness against runtime hardware effects.

To the best of our knowledge, there is no state-of-the-art practice that addresses comprehensive low-level code verification for correctness against hardware effects. Unlike our extensible framework, other works are largely restricted to specific use cases and platforms. Apparently, the comprehensive verification and performance analysis of the manually implemented low-level code is an area that needs further exploration in the research community.

8. Conclusion and future work

In this paper, we proposed a novel framework for analyzing and verifying the correctness of manually implemented low-level RTOS or BSW code against internal hardware effects. For hardware behavior analysis, we used a WCET tool (OTAWA) as a microarchitecture analyzer. Our verification and performance analysis process uses the intermediate results of WCET analysis to ensure the side-effect-free behavior of low-level code and to identify performance bottlenecks in the respective hardware architecture. In this way, our framework mitigates the need for formal models of hardware and avoids the need to analyze hardware behavior in the formal method environment. The results of our framework facilitate the construction of formal models of low-level code.

Moreover, our approach works with XGraph information that represents all potential execution paths under all possible dynamic hardware behaviors (cache hit/miss, pipeline execution). Therefore, our approach is more effective in analyzing and verifying low-level code against internal hardware effects than traditional rigorous testing methods that are test-case oriented.

In this paper, first, we described the intermediate results of the WCET analysis that are important to our verification and performance analysis process. Next, we logically deduced the side-effect-free behavior of low-level code as specifications and formalized them into a set of mathematical constraints based on the hardware specifications. We also specified and formalized the specifications to analyze performance bottlenecks and explained our process. Next, we demonstrated our framework by analyzing and verifying a real-world context switch and kernel startup code for the AURIX TriCore architecture. Finally, we performed an empirical evaluation of our process and observed a

moderate linear relationship between verification and analysis time and the size of the input state space.

During the demonstration (Section 4), we detected failed synchronization dependency checks, potential memory race conditions, and performance bottlenecks in the code sequences. Based on the functional logic, we identified that the memory race conditions do not happen. We resolved the synchronization dependencies by modifying the code sequences and re-verified them. The re-verification effort was minimal, requiring only changes to the code sequences. We captured the program flow information, verification results, execution times of the instruction and BBs, the order of instruction execution, and performance bottlenecks.

In the future, we intend to automate the manual analysis of memory race condition check results by allowing developers to provide program invariants as input to our process. We plan to extend our framework to support ARM and RISC-V architectures, including out-of-order processors, and analyze the acceptance of our process when integrated into existing development workflows. We aim to use the results of our framework and establish modeling patterns to construct formal models of low-level code. Then, we intend to combine these models with the high-level models of the software to verify the correctness against functional and non-functional requirements. In addition to the verification and performance analysis of low-level code, another application of our framework could be to analyze and verify arbitrary software binaries against their requirement specifications if we can formalize them as constraints.

CRediT authorship contribution statement

Vignesh Manjunath: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Marcel Baunach:** Supervision, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: This work has been supported by the FFG under contract No. 881844: "Pro^{2}Future" (Products and Production Systems of the Future), Graz University of Technology (TU Graz), and Elektrobit Automotive GmbH in the joint research project CompEAS. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Artifacts availability

The source code of our verification and performance analysis process, the intermediate results of the WCET analysis, and the results of the process generated during the demonstration (see Section 4) and other experimental evaluations are available in the artifacts at https://dx.doi.org/10.6084/m9.figshare.25657482.

Acknowledgments

This work has been supported by (1) the FFG under contract No. 881844: "Pro^{2}Future" (Products and Production Systems of the Future), Graz University of Technology (TU Graz), and Elektrobit Automotive GmbH in the joint research project CompEAS and (2) the TU Graz Open Access Publishing Fund.

References

[1] R.M. Gomes, B. Aichernig, M. Baunach, A formal modeling approach for portable low-level OS functionality, in: F. de Boer, A. Cerone (Eds.), Software Engineering and Formal Methods, Springer International Publishing, Cham, 2020, pp. 155–174, http://dx.doi.org/10.1007/978-3-030-58768-0_9.

[2] J. Shi, J. He, H. Zhu, H. Fang, Y. Huang, X. Zhang, ORIENTAIS: Formal verified OSEK/VDX real-time operating system, in: 2012 IEEE 17th International Conference on Engineering of Complex Computer Systems, IEEE, 2012, pp. 293–301, http://dx.doi.org/10.1109/ICECCS20050.2012.6299224.

[3] J. Shi, L. Zhu, Y. Huang, J. Guo, H. Zhu, H. Fang, X. Ye, Binary code level verification for interrupt safety properties of real-time operating system, in: 2012 Sixth International Symposium on Theoretical Aspects of Software Engineering, IEEE, 2012, pp. 223–226, http://dx.doi.org/10.1109/TASE.2012.46.

[4] E. Clarke, O. Grumberg, S. Jha, Y. Lu, H. Veith, Progress on the state explosion problem in model checking, in: Informatics, Springer, 2001, pp. 176–194, http://dx.doi.org/10.1007/3-540-44577-3_12.

[5] C. Ballabriga, H. Cassé, C. Rochange, P. Sainrat, OTAWA: An open toolbox for adaptive WCET analysis, in: IFIP International Workshop on Software Technolgies for Embedded and Ubiquitous Systems, Springer, 2010, pp. 35–46, http://dx.doi.org/10.1007/978-3-642-16256-5_6.

[6] M. Staron, D. Durisic, AUTOSAR standard, in: Automotive Software Architectures, Springer, 2017, pp. 81–116, http://dx.doi.org/10.1007/978-3-319-58610-6_4.

[7] FreeRTOS, 2023, https://freertos.org/. (Accessed 10 December 2023).

[8] P. Cousot, R. Cousot, Basic concepts of abstract interpretation, in: Building the Information Society, Springer, 2004, pp. 359–366, http://dx.doi.org/10.1007/978-1-4020-8157-6_27.

[9] H. Cassé, F. Birée, P. Sainrat, Multi-architecture value analysis for machine code, in: 13th International Workshop on Worst-Case Execution Time Analysis-WCET 2013, 2013, pp. pp–42, http://dx.doi.org/10.4230/OASIcs.WCET.2013.42.

[10] X. Li, A. Roychoudhury, T. Mitra, Modeling out-of-order processors for software timing analysis, in: 25th IEEE International Real-Time Systems Symposium, IEEE, 2004, pp. 92–103, http://dx.doi.org/10.1109/REAL.2004.33.

[11] H. Theiling, C. Ferdinand, Combining abstract interpretation and ILP for microarchitecture modelling and program path analysis, in: Proceedings 19th IEEE Real-Time Systems Symposium (Cat. No. 98CB36279), IEEE, 1998, pp. 144–153, http://dx.doi.org/10.1109/REAL.1998.739739.

[12] W.-T. Sun, E. Jenn, H. Cassé, Validating static WCET analysis: a method and its application, in: 19th International Workshop on Worst-Case Execution Time Analysis, WCET 2019, 2019, http://dx.doi.org/10.4230/OASIcs.WCET.2019.6, 6:1–6:10.

[13] TriCore TC1.6P & TC1.6E User Manual Vol 1 & 2 V1.0, Infineon Technologies AG, 2012.

[14] TC27x D-Step User Manual V2.2, Infineon Technologies AG, 2014.

[15] W.-T. Sun, E. Jenn, H. Cassé, Build your own static WCET analyser: the case of the automotive processor AURIX TC275, in: 10th European Congress on Embedded Real Time Software and Systems, ERTS 2020, 2020, URL https://hal.archives-ouvertes.fr/hal-02507130.

[16] iLLD: Infineon low-level drivers, 2023, https://www.infineon.com/cms/en/tools/aurix-embedded-sw/aurix-illd-drivers/. (Accessed 01 January 2023).

[17] Dot language, 2022, (Accessed 1 June 2022). URL https://graphviz.org/doc/info/lang.html.

[18] HTMLParser Python library, 2022, (Accessed 1 June 2022). URL https://docs.python.org/3/library/html.parser.html.

[19] Pydot: Python interface to GraphViz, 2022, (Accessed 1 June 2022). URL https://pypi.org/project/pydot/.

[20] Pandas Python library, 2022, (Accessed 1 June 2022). URL https://pandas.pydata.org/.

[21] Embench: benchmark suite, 2023, (Accessed 10 February 2023). URL https://github.com/embench/embench-iot.

[22] W. Amme, P. Braun, F. Thomasset, E. Zehendner, Data dependence analysis of assembly code, Int. J. Parallel Program. 28 (5) (2000) 431–467, http://dx.doi.org/10.1023/A:1007588710878.

[23] M. Martel, Validation of assembler programs for dsps: a static analyzer, in: Proceedings of the 5th ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering, 2004, pp. 8–13, http://dx.doi.org/10.1145/996821.996827.

[24] J. Kinder, Static Analysis of X86 Executables, Tech. rep., Technische Universität Darmstadt, 2010.

[25] C. Ballabriga, J. Forget, G. Lipari, Abstract interpretation of binary code with memory accesses using polyhedra, 2017, http://dx.doi.org/10.48550/arXiv.1711.07257, arXiv preprint arXiv:1711.07257.

[26] M. Chacko, K.P. Jacob, Validation of microcontroller codes: an architecture oriented approach, Int. J. Intell. Comput. Cybern. (2011) http://dx.doi.org/10.1108/17563781111186743.

[27] H. Aslanyan, M. Arutunian, G. Keropyan, S. Kurmangaleev, V. Vardanyan, BinSide: Static analysis framework for defects detection in binary code, in: 2020 Ivannikov Memorial Workshop, IVMEM, IEEE, 2020, pp. 3–8, http://dx.doi.org/10.1109/IVMEM51402.2020.00007.

[28] J. Brauer, R.R. Hansen, S. Kowalewski, K.G. Larsen, M.C. Olesen, Adaptable value-set analysis for low-level code, in: 6th International Workshop on Systems Software Verification, Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2012, http://dx.doi.org/10.4230/OASIcs.SSV.2011.32.

[29] A.E. Dalsgaard, M.C. Olesen, M. Toft, R.R. Hansen, K.G. Larsen, Metamoc: Modular execution time analysis using model checking, in: 10th International Workshop on Worst-Case Execution Time Analysis, WCET 2010, Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2010, http://dx.doi.org/10.4230/OASIcs.WCET.2010.113.

[30] L. Liang, T. Melham, D. Kroening, P. Schrammel, M. Tautschnig, Effective verification for low-level software with competing interrupts, ACM Trans. Embed. Comput. Syst. (TECS) 17 (2) (2017) 1–26, http://dx.doi.org/10.1145/3147432.

[31] J.-R. Abrial, Modeling in Event-B: System and Software Engineering, Cambridge University Press, 2010, http://dx.doi.org/10.1017/CBO9781139195881.

[32] J. McMahan, M. Christensen, K. Dewey, B. Hardekopf, T. Sherwood, Bouncer: Static program analysis in hardware, in: 2019 ACM/IEEE 46th Annual International Symposium on Computer Architecture, ISCA, IEEE, 2019, pp. 711–722, http://dx.doi.org/10.1145/3307650.3322256.

[33] A. Chlipala, The Bedrock structured programming system: Combining generative metaprogramming and Hoare logic in an extensible program verifier, in: Proceedings of the 18th ACM SIGPLAN International Conference on Functional Programming, 2013, pp. 391–402.

[34] A. Chlipala, Mostly-automated verification of low-level programs in computational separation logic, in: Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation, 2011, pp. 234–245.

[35] R. Gu, Z. Shao, H. Chen, X.N. Wu, J. Kim, V. Sjöberg, D. Costanzo, {CertiKOS}: An extensible architecture for building certified concurrent {OS} kernels, in: 12th USENIX Symposium on Operating Systems Design and Implementation, OSDI 16, 2016, pp. 653–669.

[36] B. Barras, S. Boutin, C. Cornes, J. Courant, Y. Coscoy, D. Delahaye, D. de Rauglaudre, J.-C. Filliâtre, E. Giménez, H. Herbelin, et al., The Coq Proof Assistant Reference Manual, INRIA, 1999, version 6 (11).



Vignesh Manjunath is a researcher at Pro2Future GmbH and a Ph.D. student in the Embedded Architectures and Systems group at the Institute of Technical Informatics at Graz University of Technology. He holds a master's degree in embedded software systems and has profound industry experience in embedded automotive software development. His main research area is the formal modeling and verification of real-time operating system kernels and basic software modules. His research focuses on promoting the long-term maintenance, portability, and correctness of these modules while taking performance into account.



Marcel Baunach is professor for Embedded Automotive Systems and head of the EAS Group at the Institute of Technical Informatics at Graz University of Technology, Austria. His research areas include hardware and software for highly dependable and sustainable embedded systems. The focus is on real-time operating systems and processor architectures, in particular on concepts for the design, verification and portability of system software, dynamic composition of modular software, as well as application-specific and reconfigurable processors. Application areas include, e.g., future vehicles, the Internet of Things or cyber–physical systems.